Q21: Composing Networks and Learned Policies

Paradise

October 2026

The question

- "How do different networks and learned RL policies combine in one system, and what is the design intuition?"
- Observation: no deployed system is one network on one loss
- Answer preview: **five patterns, one trade-off, three axes**

Five composition patterns

Pattern	**Seam**
Pipeline	upstream output feeds downstream
Ensemble	votes / average (all fire)
Mixture-of-experts	learned router picks experts
Options / hierarchy	high-level policy picks sub-policies
LLM orchestrator	model calls tools/components

Each pattern = a different answer to "who decides, and what crosses the seam?"

MoE: composition inside one training run

- 1991: gating network allocates cases to experts [5]
- 2017: sparsely-gated layer, 2 of 137B experts per token [9]
- 2022: Switch top-1 routing to a trillion parameters [4]
- Why it works: the joints (router–expert) are *jointly trained* $\Rightarrow$ no interface brittleness
- Constraint: all experts share architecture, data stream, loss

Options: composition along time

- Options = closed-loop policies + initiation set + termination [10]
- Fixed option set $\Rightarrow$ the high level faces a semi-MDP
- MAXQ: value function decomposes across the hierarchy [3]
- Option-critic: learn the options too [2]
- Payoff: state abstraction; cost: credit for delayed reward must split across options

LLM as orchestrator

- ReAct: interleave reasoning traces with actions [11]
- Toolformer: model learns to emit API calls self-supervised [7]
- SayCan: $p(\text{skill} \mid \text{instruction}) \times p(\text{success} \mid \text{state})$ from a learned value function [1]
- MuZero: a different orchestrator — search over a learned model [8]

When composition wins — and loses

Wins:

- **Data heterogeneity:** components train where their data lives
- **Observation rates differ:** options decide at boundaries, skills every step
- **Reusability:** add a skill = add a description + value function [1]
- **Debuggability:** unit-test each component; readable traces [11]

Loses:

- **Compounding error:** k stages, success $\sim p^k$; the end-to-end counter-argument [6]
- **Interface brittleness:** retrain the producer, silently break the consumer
- **Credit assignment:** who was wrong? gradients don't cross discrete seams

Trade: learning-theoretic optimality $\leftrightarrow$ engineering scalability

The interface problem

What crosses the seam?

1. **Embeddings:** max bandwidth, differentiable, unstable, silent failure
2. **Typed APIs:** checked contracts, loud failure, someone designs them [7]
3. **Natural language:** universal, readable, lossy, hallucination-prone [1]

Interface choice = where gradient can flow

Three design axes

- **Latency:** router $\ll$ pipeline $<$ LLM-per-step; search has a budget [8]
- **Trainability of joints:** gradients across seams? embeddings vs. discrete choices
- **Observability:** can you localize a 3 a.m. failure?

The axes conflict: highest bandwidth = least observable. Design = negotiating the three.

Toy demo (code/)

- 4-expert mixture, learned router, clustered synthetic data
- vs. monolith with *more* parameters (316 vs. 180) — conservative
- Router learns cluster assignment (100%, up to permutation)
- Composed system beats monolith on held-out loss (0.029 vs. 0.066)
- Kill the router (random routing) $\Rightarrow$ loss explodes to 10.2

Takeaway

Composition buys reusability, heterogeneous-data compatibility, and debuggability at the price of compounding error, brittle interfaces, and cross-component credit assignment — the design skill is placing each seam on the latency / trainability / observability surface.

References I

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al.

Do as i can, not as i say: Grounding language in robotic affordances.

arXiv preprint arXiv:2204.01691, 2022.

Pierre-Luc Bacon, Jean Harb, and Doina Precup.

The option-critic architecture.

In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, pages 1674–1680, 2017.

Thomas G. Dietterich.

Hierarchical reinforcement learning with the maxq value function decomposition.

Journal of Artificial Intelligence Research, 13:227–303, 2000.

William Fedus, Barret Zoph, and Noam Shazeer.

Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity.

Journal of Machine Learning Research, 23(120):1–39, 2022.

References II


Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton.
Adaptive mixtures of local experts.
Neural Computation, 3(1):79–87, 1991.


Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel.
End-to-end training of deep visuomotor policies.
Journal of Machine Learning Research, 17(39):1–40, 2016.


Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom.
Toolformer: Language models can teach themselves to use tools.
In *Advances in Neural Information Processing Systems*, 2023.


Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver.
Mastering atari, go, chess and shogi by planning with a learned model.
Nature, 588:604–609, 2020.

References III



Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean.

Outrageously large neural networks: The sparsely-gated mixture-of-experts layer.

In *International Conference on Learning Representations*, 2017.



Richard S. Sutton, Doina Precup, and Satinder Singh.

Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning.

Artificial Intelligence, 112(1–2):181–211, 1999.



Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.

React: Synergizing reasoning and acting in language models.

In *International Conference on Learning Representations*, 2023.